מבוא למדעי המחשב - הרצאה 2

הערה

סיכום קצר

- **הנושא המרכזי:**
- **הרעיון המרכזי / מה לקחת מההרצאה:**

1. מושגים והגדרות מרכזיים

חשוב

הגדרות ותכונות נשמרות

2. סיכום ההרצאה

נושא 1

הערה

הערות והסברים

Negative Numbers

there are three ways of representing negative numbers in the binary system.
the usual way of doing so is by "Two's complement".
the three ways are:
1)

Sign & magnitude: 1-bit sign

MSB (most significant Bit) is a sign bit:

- 0 - positive
- 1 - negative




```
|MSB | MAGNITUDE BITS                    |
```

the problem is that 0 can be represented by 2 different bit numbers, for example: 00000000 and 10000000.
another problem is that addition doesnt work:
0000 0101 =5
1000 0101 =-5
their addition gives us:
1000 1010 = -10

2.

One's Complement

flip every bit in X to make it into -X. 1 becomes 0 and 0 becomes 1
example in 4 bits:
0111 = 7
1000 = -7

0 still becomes a problem here, it can be represented both by 0000 and 1111.
we also still get that MSB will be equal 1 and the negative numbers and equal to 0 in the positive numbers.

addition works perfectly in the One's Complement method!
0101=5
1010=5
add them together:
1111 = 0

0101= 5
1100= -3

add them up:
1 0001
-> 1
0010 = 2

if we have a carry of 1 -> we will add it. (end around carry)
it acts as the field $\mathbb{Z}_{15}$ with modulo 15

2-1:
0000 0010
1111 1110
| |
1 0000 0000
0000 0001 =1

-7+3
1111 1000
0000 0011
| |
1111 1011 = -4

3.

Two's Complement

המשלים של אותו מספר עם n ביטים ביחס למספר 2^n
how to calculate: flip all bits and add 1

0010 = 2 (1101+1=1110)
1110 =-2
instead of only having all numbers throughout -7 and 7 like in the other 2 methods (since 0 took 2 numers), for 4 bits, we get -8 as 1000, tho we dont get a positive 8.

heres an example of how the complementary of 3 with respect for 2^8 is calculated:
1 0000 0000 (2^8)

0000 0011 (3)

1111 1101 (-3)

how to convert decimal (regular number) to binary:

positive - do as normal

negative - return it to positive, find its number in bits, flip those bits and then add 1.

how addition works:

0101 = 5

1101 = -3

1 0010 =2

we ignore the carry if it exists.

converting back from binary to decimal

conversion process:

if MSB is 0 - do regular conversion.

if MSB is 1 - flip all bits -> add 1 -> do regular conversion and add "-" in the back

use equation:

$$number = -b_{n-1} \times 2^{n-1} + \sum_{i=0}^{n-2} b_i \times 2^i$$

$$\left| b_{n-1} \right| b_n \left| \dots \right| \left|$$

representation range in n bits

positive numbers: $x \in [0, 2^n - 1]$

1's complement: $x \in$

continue later

overflow

the number is too large to be represented in the given number of bits.
0000 0101 (5)
+
0111 1111 (127)

=
10000100 (-124) -> 5+127=132 $\notin [-128, 127]$ (meaning between -128 and 127 including those numbers)
we added 2 positive numbers and got a negative numbers.

underflow

the number is too small to be represented in the given number of bits.
1011 0000 (-80)
+
1100 1100 (-52)

=
0111 1100 (124) -> -52-80 =-132 $\notin [-128, 127]$ (meaning between -128 and 127 including those numbers)

Floating Points

representing fractions and very large negative or positive numbers
IEEE standart

normal form = a single non-zero digit before the dot.

how it looks in decimal

$31.4159 = +3.14159 \times 10^{+1}$
$-0.00132 = -1.32 \times 10^{-3}$

how it looks in binary:

$$1010.01_2 = +1.01001 \times 2^3{}_2$$
$$0.00010101_2 = +1.0101 \times 2_2^{-4}$$

look at the right side (after the =)
the plus is called a a sign
the 1 is called the mantissa
the power of the number (in this case , 2) is called the exponent.

in 32bits:

sign

takes up 1 bit, 1 for + and 0 for -

exponent

takes up 8 bits, can be any number from -127 to 128 (where 0 is represented as 127, for example 3 would me 130), to calculate it back, calculate the value of the bits, and substitute 127 from the answer.

mantissa

takes up 23 bits, come after the 1 (1.(mantissa)x2^x), can be any number within the 23 bit constrains. also the 1 in the beginning is unchangeable since we are using bits and we specified that the number must to not be equal 0 so it will always be 1.



Translating back to ordinal numbers

insert picture

Converting floating decimal to binary

-10.3_{10}

keep the sign and the convert the whole part to binary using conversion algorithm for integers:

10 = 1010

fractional conversion - decimal to binary

multiply by 2, take the whole part, repeat until one of 2 things happen:

1. reach a periodic infinite calculation

-1010.010011001100....

in normal form: -1.010010011001100x2^3

exponent: 8+127=135 = 10000111

(we ask ourself how my times do i need to move the dot, and to which direction? since 431 is represented by 9 bits, we need to move the dot 8 times so it would be (1.rest of the 8 bits) and the dot needs to move to the left, so it would be positive therefore we add 127.)

110100101 = 421

$$0.4 \cdot 2 = 0.8 \rightarrow 0$$
$$0.8 \cdot 2 = 1.6 \rightarrow 1$$
$$0.6 \cdot 2 = 1.2 \rightarrow 1$$
$$0.2 \cdot 2 = 0.4 \rightarrow 0$$
0.4חוזר חלילה

1	10000111	10100101011001100110011

in a double type, the ,mantissa gets 52 bits, exponent gets 11 bits and the sign still gets 1 bits since its only a 1 or a 0.

Data types

data types define:

- the kind of data a variable can hold
- how much memory it occupies

- how the system interprets that data

primitive data types

- int, char, float, long, double, (unsigned int, unsigned char)=>only positive numbers, ...

```
int x = 2;
```

a number, usually 32 bits in memory.

- variable named x
- its data type is int
- it is initialized with the value 2
- when the program will run, it will be stored in the RAM (probably in 4 bytes)

in general a variable always has:

1. name
2. type
3. address
4. value

- name, type, address - never change, value may change
- if value hasn't been defined/assigned, the variable's value is "garbage"(Null)
- a variable must be declared before it is used
  we can add "const"(constant) before int to lock its value into place, making it unchangeable.

Types of operators

- assignment (=) -> can assign value, compare values in ifs etc...
- arithmetic => + - * / %
- Bitwise => and or XOR not shift left shift right

תשובות

001

בינארי 0011

ONES COMPLEMENT 1100